

API_Implementation_Requirements

Hospital Directory Read API — Implementation Requirements

Prepared for: External data company implementing the production API
Status: Approved baseline — resolves the review round on the Patient resource and Health response
Companion file: Hospital_Directory_API_OpenAPI.yaml (machine-readable contract — authoritative on conflicts, see section 9)
Reference implementation: A working mock API implementing this exact contract is available for the vendor to test against during development (FastAPI + PostgreSQL, source data fictional).

0. Purpose and scope

This API gives two internal hospital applications (HCAT and HCopilot) controlled, **read-only** access to three categories of hospital directory data: patients, doctors, and workers. It is a stable boundary in front of the hospital's internal data system — the consuming applications never connect to that system directly, and the vendor's implementation never needs to expose more than what is defined here.

Out of scope for this API (do not build these): creating, updating, or deleting any record; complaint/incident data; HCAT or HCopilot application data; analytics or reporting; admin screens; user login; API-key management screens.

A mock implementation of this same contract already exists and is used for HCAT/HCopilot development today. The production API the vendor builds must be **externally indistinguishable** from the mock — same paths, parameters, field names, types, status codes, and authentication behavior. Only the data source differs. See section 9 for how conflicts between documents are resolved.

1. Required resources

Resource	Endpoint base	Purpose
Patients	/api/directory/v1/patients	Search and retrieve patient records. Each result represents one person — a patient does not repeat across rows.
Doctors	/api/directory/v1/doctors	Search and retrieve individual doctors.
Workers	/api/directory/v1/workers	Search and retrieve individual hospital workers/employees.
Health	/api/directory/v1/health	Report service availability. Not a business resource — no API key required.

All endpoints are GET only. POST / PUT / PATCH / DELETE on any path under /api/directory/v1 must return 405 Method Not Allowed.

Base path /api/directory/v1 is fixed for this version. A future breaking change gets a new path (/api/directory/v2); it must never silently change what v1 returns.

2. Required fields for every resource

Identifier rule (applies to every resource): every externally-visible identifier (patient_id , doctor_id , employee_id) is a JSON **string**, even if the source database stores it as an integer. This avoids problems with leading zeros, alphanumeric values, prefixes, and very large numbers. Identifiers are for identity, never for arithmetic.

Date/time rule (applies to every resource): dates use YYYY-MM-DD ; date-times use ISO 8601 with a timezone offset (YYYY-MM-DDTHH:MM:SS±HH:MM). A field with no value is JSON `null` — never an empty string, and never a placeholder date such as 1900-01-01 or 1970-01-01 .

Boolean rule: booleans are real JSON booleans (`true` / `false`), never the strings `"Y"` / `"N"` / `"1"` / `"0"` .

2.1 Patient

Field	Type	Required	Nullable	Notes
patient_id	string	Yes	No	Permanent, stable identifier for the person.
full_name	string	Yes	No	Display name; must support Arabic and English without corruption.
first_name	string	No	Yes	
last_name	string	No	Yes	
birth_date	date	No	Yes	Preferred source for accurate age.
age	integer	No	Yes	May be supplied directly, or derived from <code>birth_date</code> — confirm which the source system provides.
sex	string	No	Yes	Allowed values to be confirmed by the source-system owner (see the open items list, section 9).

Patients are modeled as a **person**, not a visit. `visit_id` , `phone_number` , `medical_file_number` , `document_number` , `arrival_time` , and `departure_time` are **not** part of this resource — none of them have a well-defined, non-visit-scoped meaning at the person level, so rather than force a loose approximation (e.g. "most recent visit's arrival time") onto a person-level record, they're dropped entirely. If a future need for visit-level data emerges, it will be its own explicit resource rather than bolted onto Patient.

Accepted trade-off: with those fields gone, two patients who share a `full_name` are only distinguishable in search results by `patient_id` , `birth_date` , and `age` . This is intentional, not an oversight — see section 5.

2.2 Doctor

Field	Type	Required	Nullable	Notes
doctor_id	string	Yes	No	Stable identifier.
full_name	string	Yes	No	
specialty_id	string	No	Yes	Expose only if available.
specialty_name	string	No	Yes	
department_id	string	No	Yes	Include only if an authoritative source exists. This is the vendor's own department identifier space, not the hospital application's internal one — the two are not expected to match (see open item in section 9 about hierarchy).
department_name	string	No	Yes	Include only if an authoritative source exists. Same note as <code>department_id</code> .
is_active	boolean	Yes	No	Whether the record is currently eligible for selection in new workflows — not whether the doctor is on duty right now. Inactive doctors are excluded from default search but must remain retrievable by exact <code>doctor_id</code> .

2.3 Worker

Field	Type	Required	Nullable	Notes
employee_id	string	Yes	No	Stable identifier.
full_name	string	Yes	No	
job_id	string	No	Yes	Low priority for the hospital application today, but not removed from the contract.

Field	Type	Required	Nullable	Notes
job_title	string	No	Yes	
department_id	string	No	Yes	
section_id	string	No	Yes	
administration_id	string	No	Yes	
is_manager	boolean	No	Yes	Display-only.
is_active	boolean	Yes	No	Same meaning as the doctor field above. Inactive workers excluded from default search, retrievable by exact employee_id .

2.4 Health

Minimal by design — this endpoint exists only so monitoring can check reachability without a credential (see section 8). Detailed diagnostics are intentionally out of scope for it.

Field	Type	Required
status	string, "healthy" "unhealthy"	Yes

The vendor may include additional fields (service name, version, timestamp) if convenient for their own operations, but only status is part of the contract HCAT/HCCopilot depend on. The reference mock returns status only.

3. Search parameters for every endpoint

3.1 GET /patients

Parameter	Required	Description
q	Conditional	General search text — name or ID.
patient_id	Conditional	Exact patient identifier filter.
limit	No	Default 100, min 1, max 500.
offset	No	Default 0, min 0.

At least one of q, patient_id is required. A request with neither returns 400 Bad Request (MISSING_SEARCH_CRITERIA) — patient enumeration without any criterion is not supported.

q should match against whatever of these the source system can search: patient ID, full name. Consuming applications wait for at least 2 characters before firing a free-text name search, but this is a client-side convention, not a rule the API enforces — the API accepts any non-empty q .

3.2 GET /patients/{patient_id}

Path parameter required. No query parameters. Returns exactly one patient — never a list, never related patients or family members.

3.3 GET /doctors

Parameter	Required	Description
q	No	Matches doctor ID, full name, specialty name, department name where available.
active_only	No	Boolean, default true .
limit / offset	No	Same pagination rules as patients.

Unlike patients, doctors may be called with **no** search term at all — it returns a paginated list of active doctors by default.

3.4 GET /doctors/{doctor_id}

Path parameter required, no query parameters. `active_only` does **not** apply here — inactive doctors remain retrievable by exact ID.

3.5 GET /workers

Parameter	Required	Description
q	No	Matches employee ID, full name, job title, department/section/administration IDs where available.
active_only	No	Boolean, default <code>true</code> .
limit / offset	No	Same pagination rules as patients.

3.6 GET /workers/{employee_id}

Same shape as 3.4 — path parameter required, `active_only` does not apply.

3.7 GET /health

No parameters, no authentication.

4. Response object structures

4.1 List envelope (patients, doctors, workers search)

Every list endpoint returns this shape, regardless of resource:

```
{
  "success": true,
  "items": [ /* array of resource objects, see section 2 */ ],
  "total": 243,
  "limit": 20,
  "offset": 40
}
```

`total` is the count of matching records **before** pagination is applied — not the length of `items`.

4.2 Single-object responses (exact lookups)

GET /patients/{patient_id}, GET /doctors/{doctor_id}, and GET /workers/{employee_id} return the bare resource object directly — **no** envelope, no wrapping `success` / `items` keys. Example:

```
{
  "patient_id": "P-10025",
  "full_name": "Ahmad Ali",
  "first_name": "Ahmad",
  "last_name": "Ali",
  "birth_date": "1980-05-12",
  "age": 46,
  "sex": "M"
}
```

4.3 Content type

Every response — success or error — must be

Content-Type: application/json; charset=utf-8. The `charset=utf-8` is not optional: Arabic text corruption is treated as a contract violation, not a display bug.

5. Duplicate-name handling

Hospital data will contain people who share the same `full_name` — this is expected, not an error condition, and the API must never attempt to detect, merge, or deduplicate records based on name similarity.

Rules:

- **Identity is always the ID, never the name.** `patient_id`, `doctor_id`, and `employee_id` are the only fields that uniquely identify a record. `full_name` is a display/search convenience field only.
- **Search returns every matching row independently.** If three doctors are all named "Dr. Ahmad Ali", a search for `q=ahmad ali` returns all three as separate items, each with its own `doctor_id`. The API does not pick a "best match" or collapse them.
- **Disambiguation is a consuming-application concern, not an API concern.** The API's job is to return enough distinguishing fields (ID, specialty/department for doctors, job title/department for workers, birth date/age for patients) that a human user or the calling application can tell the records apart and select the correct one. The API itself must never guess. For patients specifically, this now means `patient_id + birth_date / age` only (section 2.1) — a deliberate, accepted trade-off, not a gap to fix.
- **Exact lookup by ID is unambiguous by construction.** Once a specific `patient_id`, `doctor_id`, or `employee_id` is chosen from search results, the exact-lookup endpoints return exactly one record — duplicate names are no longer a concern at that point.

6. No-result handling

No-result behavior differs by endpoint type — this distinction matters and must not be conflated:

- **List/search endpoints** (`GET /patients`, `GET /doctors`, `GET /workers`) — a search that matches nothing is **not an error**. Return `200 OK` with:

```
{ "success": true, "items": [], "total": 0, "limit": 100, "offset": 0 }
```

Never return `404` for an empty search result.

- **Exact-lookup endpoints** (`GET /patients/{patient_id}`, `GET /doctors/{doctor_id}`, `GET /workers/{employee_id}`) — a record that does not exist for the given ID is a `404 Not Found`, with the matching error code from section 7 (`PATIENT_NOT_FOUND`, `DOCTOR_NOT_FOUND`, `WORKER_NOT_FOUND`). This is different from "the service is unavailable" (`503`) — see section 7.
- **A malformed/missing search parameter is different from "no results."** `GET /patients` with no `q/patient_id` at all is `400 Bad Request` (`MISSING_SEARCH_CRITERIA`), not an empty `200`. The request itself is invalid, independent of whether any data would have matched.

7. Error handling

All errors return this JSON shape, with `Content-Type: application/json; charset=utf-8`:

```
{
  "error": "PATIENT_NOT_FOUND",
  "message": "The requested patient was not found"
}
```

`error` is a stable machine-readable code consuming applications can branch on; `message` is a human-readable explanation and may change wording without being a breaking change. Never return a raw stack trace, SQL error, or internal exception detail in either field.

7.1 Status codes and error codes

Status	When	error code
400	Patient search called with no search criterion	MISSING_SEARCH_CRITERIA
401	Missing or invalid X-API-Key	UNAUTHORIZED
404	Exact patient not found	PATIENT_NOT_FOUND
404	Exact doctor not found	DOCTOR_NOT_FOUND
404	Exact worker not found	WORKER_NOT_FOUND
405	Unsupported HTTP method on a valid path	METHOD_NOT_ALLOWED
422	Parameter present but invalid (e.g. limit=9999, out of the 1–500 range)	VALIDATION_ERROR
500	Unexpected server error	INTERNAL_SERVER_ERROR
503	API is running but its data source is unreachable	(see health endpoint, section 2.4)

These exact `error` string values are part of the contract — the reference mock implementation returns precisely these codes, and the production API must match them so HCAT/HCCopilot's error-handling logic works unmodified against either.

7.2 The 404-vs-503 distinction (critical)

These are different outcomes and must never be conflated:

- **404** — the API responded normally; the record genuinely does not exist.
- **503** (or a connection failure) — the API could not reach its data source. This must never be reported to the consuming application as "not found." Consuming applications are required to preserve this distinction on their side too, so the production API must make it possible to distinguish them in the first place.

8. Authentication requirements

- Every endpoint except `GET /health` requires the header:

```
X-API-Key: <shared-secret>
```

- Missing or invalid key → `401 Unauthorized`, `error: "UNAUTHORIZED"`.
- Version 1 uses **one static, shared key** — both HCAT and HCCopilot use the same value. No per-application keys, no key-issuance endpoint, no key-management UI, no OAuth/session/login flow of any kind.
- The key must be supplied via runtime configuration on the vendor's side (environment variable, secret store, etc.) — never hardcoded in source and never committed to version control.
- `GET /health` is deliberately the one public endpoint, so monitoring systems can check availability without holding a credential. It must never expose business data, credentials, connection strings, or internal topology regardless of authentication.

9. OpenAPI specification

The companion file `Hospital_Directory_API_OpenAPI.yaml` is the machine-readable, authoritative version of everything above — paths, parameters, schemas, required/nullable fields, and response codes are all formally defined there, and matches this document (Patient as a person-level resource, minimal Health response). Use it to generate server stubs/client SDKs and to validate responses in CI.

Precedence when documents disagree:

1. Hospital_Directory_API_OpenAPI.yaml (wins on any conflict)
2. This document
3. Any other prose explanation given verbally or in email

Open items — raise these with the hospital's team before deviating from the published contract, rather than guessing:

1. Allowed values for `sex`.
2. Whether `age` is stored directly by the source system or must be derived from `birth_date`.
3. Whether doctor/worker `department_id` / `department_name` have an authoritative source on the vendor's side.
4. Whether the vendor can additionally expose the full department/ administration hierarchy (names, structure, not just IDs) in a form the hospital application could import — today the hospital application's own reporting has no local concept of specialty/department to group by, so doctor/worker report grouping is planned to lean on whatever hierarchy data the vendor's API can provide. This does not require a new field on Doctor/Worker; it may be a separate, later addition to the contract once scoped.
5. Final production host, port, and HTTP/HTTPS arrangement.

If implementing this contract surfaces a field that cannot be reliably mapped from the hospital's real data source, or a required decision this document doesn't cover, stop and raise it before deviating from the published contract.